

Real Estate Property & Rental Ledger System

Database Project Report

A. Project Identity

Field	Detail
Team Name	Hiền, Đạt, Kha
Team Members	Lại Thu Hiền (n24dece069@student.ptithcm.edu.vn); Lê Tuấn Kha (n24dece075@student.ptithcm.edu.vn); Nguyễn Thành Đạt (n24dece061@student.ptithcm.edu.vn)
Project Title	Real Estate Property & Rental Ledger

1. Introduction & Project Scope

1.0 Problem Statement

Small and medium-sized real estate brokerages and property management agencies that still rely on manual methods such as physical records, spreadsheets, and disconnected communication channels face several recurring operational problems. Property information and statuses are difficult to synchronize, which can result in double bookings, conflicting transactions, and inaccurate availability information. Customer data is also scattered across different records, making it difficult to maintain complete interaction histories and causing potential leads to be overlooked.

In addition, manually calculating rent, incidental fees, and broker commissions increases the risk of financial errors. The absence of automated notifications for expiring contracts and overdue payments further reduces operational efficiency and may lead to delayed actions.

The Real Estate Management System (REIMS) is designed to address these problems through a centralized relational database and management platform covering property inventory, customer relationships, property listings, viewings, leases, rental payments, maintenance requests, and financial information. The system provides the following core capabilities:

- **Property Inventory Digitization & Synchronization:** Maintains centralized property information and updates property statuses in real time to prevent conflicting transactions and duplicate deals.
- **Centralized Customer Relationship Management (CRM):** Stores customer profiles, customer requirements, property viewing schedules, and consultation histories in one centralized system.
- **Contract & Financial Automation:** Supports lease management, recurring rental billing, payment tracking, broker commission calculation, and notifications for expiring leases or overdue payments.
- **Operational Data Visualization:** Provides management dashboards for occupancy rates, cash flow, property status, and agent sales performance to support operational decision-making.

1.1 System Objective

This project designs a centralized Real Estate Management System (REIMS) that digitizes and optimizes the operational workflow of small and medium-sized brokerages and property management agencies.

The system must maintain property information and synchronize property statuses in real time, manage customers and property viewings, control listings and lease records, automate recurring financial records, and preserve complete financial histories. The system is also intended to reduce duplicate transactions, minimize manual calculation errors, improve contract and payment monitoring, and provide management with transparent operational and financial information.

The target is to replace fragmented paper-and-spreadsheet workflows with a structured system that can be queried and operated directly by the application while maintaining data consistency between properties, customers, agents, leases, payments, and maintenance activities.

1.1.1 Actor Definitions (Scope & Business Context)

The actors below are defined in the specific context of this REIMS project to clarify their responsibilities and operational scope.

- **Admin / Property Manager:** The person responsible for managing the overall property portfolio and supervising business operations. Manages property information, oversees agent allocations, supervises and processes maintenance requests, approves lease and sale contracts, and controls the general financial ledger.
- **Agent (Broker / Operations):** A real estate agent responsible for assigned property listings and related operational activities. May create and publish sale or rental listings only for assigned properties, prepare lease and purchase agreements, and record maintenance requests from tenants.
- **Owner:** The property owner or consigning owner represented in the system. Can monitor property statuses, contract histories, and maintenance events, and can reconcile received payouts and revenue streams against the financial records maintained by the system.
- **Tenant / Buyer:** The customer who rents or purchases a property through the system. A Tenant may sign a lease, make recurring rental payments, and submit maintenance requests during the tenancy. A Buyer may search for properties and proceed with a purchase transaction.

Functional Requirements (Actor-Action-Data Format)

Admin / Property Manager

1. The system must allow an Admin / Property Manager to create, update, and manage property records, including property type, location, ownership, listing purpose, and current status.
2. The system must allow an Admin / Property Manager to assign and manage Agents responsible for specific properties.
3. The system must allow an Admin / Property Manager to review and approve lease and sale contracts.
4. The system must allow an Admin / Property Manager to manage maintenance requests and update their processing status.
5. The system must allow an Admin / Property Manager to manage and review financial ledger information, including rental payments, revenue, and related transactions.
6. The system must provide an Admin / Property Manager with dashboards showing property occupancy, property status, cash flow, and agent sales performance.

Agent

7. The system must allow an Agent to create and manage listings only for properties currently assigned to that Agent.
8. The system must allow an Agent to publish sale or rental listings containing the information required to offer a property, including property type, location, and applicable price.
9. The system must allow an Agent to schedule and manage property viewings for customers.
10. The system must allow an Agent to prepare and manage lease or purchase agreement information for assigned transactions.
11. The system must allow an Agent to create and record maintenance requests submitted by tenants.

Owner

12. The system must allow an Owner to view the current status of their properties.

13. The system must allow an Owner to review property contract histories and maintenance events.
14. The system must allow an Owner to review payouts and revenue information associated with their properties.

Tenant / Buyer

15. The system must allow a Tenant / Buyer to search and view available properties.
16. The system must allow a Tenant / Buyer to book property viewings.
17. The system must allow a Tenant / Buyer to sign or participate in the applicable lease or purchase transaction.
18. The system must allow a Tenant to view rental payment information and make recurring rental payments.
19. The system must allow a Tenant to submit maintenance requests for the rented property.

Automated System Functions

20. The system must automatically synchronize property status when a lease becomes active, ends, or is terminated.
21. The system must automatically generate recurring rental invoices or payment records according to the lease payment schedule.
22. The system must automatically calculate applicable rental amounts and broker commissions based on the relevant transaction information.
23. The system must automatically notify responsible users of expiring leases and overdue payments.
24. The system must prevent conflicting active listings, overlapping confirmed viewings for the same Agent, and multiple active leases for the same property.

Non-Functional Requirements (System & Database Constraints)

NFR-01 — Data Integrity: The database must enforce primary keys, foreign keys, unique constraints, and business constraints to prevent duplicate, conflicting, or orphaned property, listing, viewing, lease, payment, and maintenance records.

NFR-02 — Real-Time Status Synchronization: Property status changes associated with leasing, maintenance, and transaction activities must be reflected consistently throughout the system so that outdated availability information does not result in conflicting transactions.

NFR-03 — Financial Accuracy: The system must enforce valid financial values, including positive rental amounts and payment amounts, and must calculate recurring rental charges and broker commissions consistently according to the stored transaction data.

NFR-04 — Historical Traceability: Financial records associated with completed or terminated leases must be retained as historical records. The system must preserve the complete financial history of a property across multiple leases.

NFR-05 — Timely Notification: The system must provide timely alerts for important events such as lease expiration and overdue payments so that responsible users can take appropriate action.

NFR-06 — Operational Visibility: The system must provide up-to-date dashboards and reports for property occupancy, property status, cash flow, and agent performance to support management activities.

NFR-07 — Access Control: System functions and data access must be limited according to the actor's responsibility. Admin / Property Manager, Agent, Owner, and Tenant / Buyer must only access and modify information relevant to their operational scope.

1.2 Business Rules & Constraints

Property & Owner

BR-01: Each property is uniquely identified by a `property_id` and must be associated with exactly one Owner. An Owner may own multiple properties.

BR-02: Each property must be classified as either `FOR_SALE` or `FOR_RENT` when it is listed. A property cannot be simultaneously listed for both purposes.

BR-03: The property status must be constrained to `AVAILABLE`, `LEASED`, or `UNDER_MAINTENANCE`.

BR-04: A property can only be leased when its status is `AVAILABLE`. Once a lease becomes active, the property status must change to `LEASED`.

BR-05: A property with status `UNDER_MAINTENANCE` cannot be leased or assigned to a new rental transaction until the maintenance request has been completed.

Agent Allocation & Property Listing

BR-06: An Agent may manage multiple properties, but each property is assigned to at most one responsible Agent at a time. The current assignment is stored directly on the property record (`PROPERTY.AgentID`); reassigning a property to a different Agent simply updates this attribute.

BR-07: An Agent may only publish or update the sale/rental offering details (`ListingType`, `Price`, and `description`) of properties currently assigned to that Agent.

BR-08: Each property listing must contain the essential information required for the property to be offered for sale or rent, including property type, location, and applicable price.

BR-09: Since `ListingType` is a single attribute on the `PROPERTY` record (see BR-02), a property can only ever have one active transaction purpose at a time; there is no separate listing history to conflict.

Lease Management

- **BR-10:** Each lease must reference exactly one Property, one Customer as the Tenant, and one responsible Agent.
- **BR-11:** A property may have multiple lease records throughout its lifetime, but it can have at most one active lease at any given time.
- **BR-12:** A lease must contain a valid start date, end date, rental amount, and payment schedule. The rental amount must be greater than zero and the end date must be later than the start date.
- **BR-13:** A lease becomes active only after the required approval process is completed and the associated property is `AVAILABLE`.
- **BR-14:** When an active lease ends or is terminated, the property's status must be updated from `LEASED` to `AVAILABLE`, unless the property is immediately placed under maintenance.

Rental Ledger & Financial History

- **BR-15:** Each active rental lease may generate multiple recurring rental invoices or payment records, and each financial record must reference exactly one lease.
- **BR-16:** Each rental payment must record the payment amount, payment date, and payment status. The payment amount must be greater than zero.
- **BR-17:** Financial records associated with completed or terminated leases must be retained as historical records and cannot be hard-deleted, even after the property becomes available again.

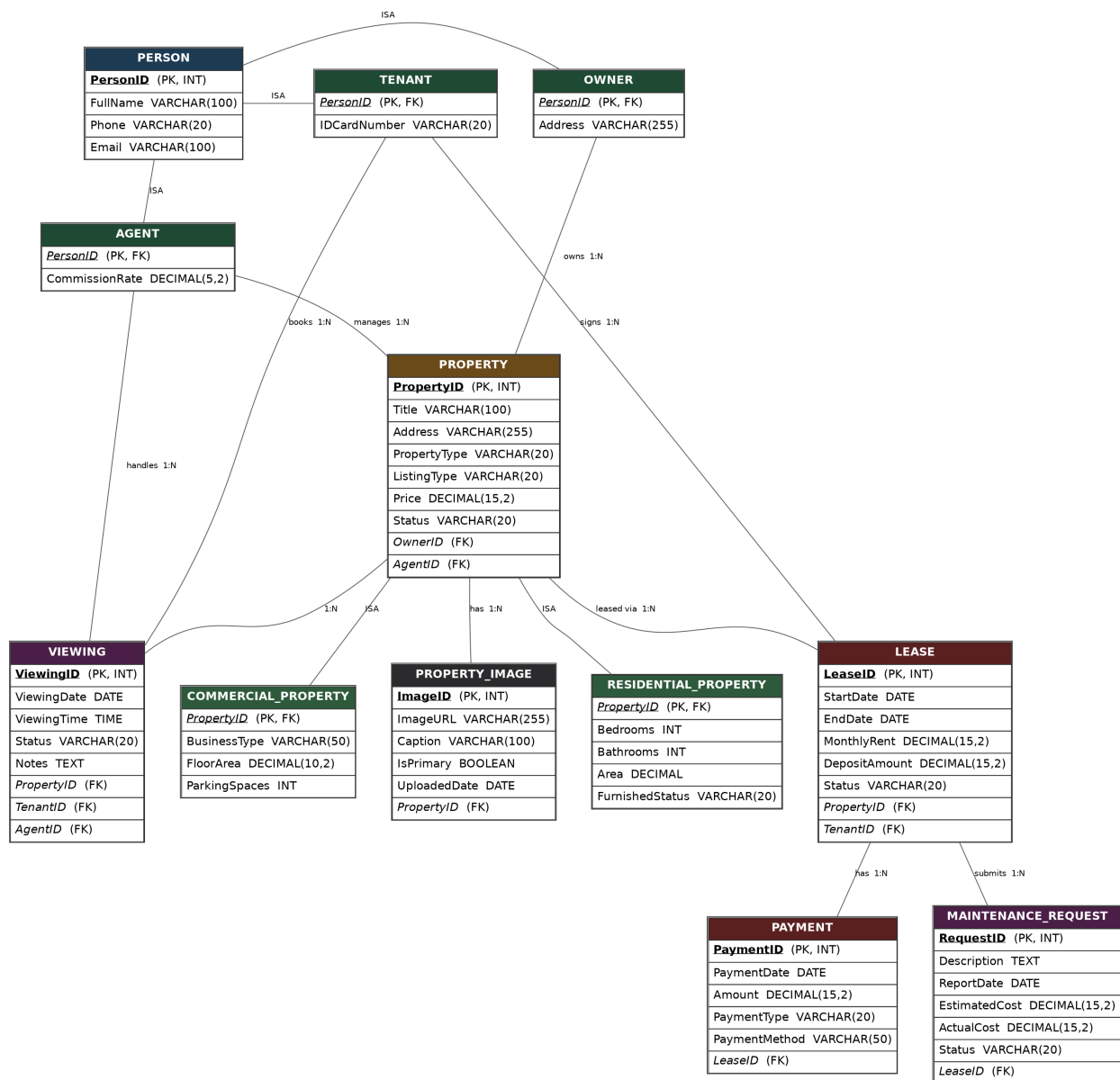
- **BR-18:** The rental ledger must preserve the complete financial history of a property across multiple leases, including previous rental payments and related financial transactions.

Maintenance Management

- **BR-19:** A Tenant or an authorized Agent may create a maintenance request on behalf of the tenant's active lease. Each maintenance request must reference exactly one Lease (which identifies the property and tenant involved) and record its description, request date, and status. Owners may view maintenance events but do not create requests directly.
- **BR-20:** When a property enters maintenance, its status must change to UNDER_MAINTENANCE. The property cannot be leased while under maintenance, and after the maintenance request is completed, the property may return to AVAILABLE if no active lease exists.

2. Database Design (ISO/IEC 19505 / IE Standards)

2.1 Conceptual Model (ER/EER Diagram)



2.1.1 ENTITY DEFINITIONS

Each entity in the EER diagram above represents the following business concept:

- **PERSON:** A general record for any individual in the system, storing name, phone, and email. Every owner, tenant, and agent is a person (BR-10).
- **OWNER:** Subtype of PERSON who owns one or more properties. Each property has exactly one owner (BR-01).
- **TENANT:** Subtype of PERSON who books property viewings and signs leases. Each tenant has a unique ID card number (BR-10, BR-11, BR-14).
- **AGENT:** Subtype of PERSON who manages properties and handles viewings, with a commission rate for closed deals. An agent can only manage the sale/rental details of properties currently assigned to them (BR-06, BR-07, BR-11).
- **PROPERTY:** A real estate unit offered for sale or for rent. It has one owner, one responsible agent, a price, and a status of AVAILABLE, LEASED, or UNDER_MAINTENANCE (BR-01 to BR-05, BR-08, BR-09).
- **RESIDENTIAL_PROPERTY:** Subtype of PROPERTY used for living, such as an apartment or a house, with details like bedrooms, bathrooms, area, and furnished status.
- **COMMERCIAL_PROPERTY:** Subtype of PROPERTY used for business, such as an office, a shop, or a warehouse, with details like business type, floor area, and parking spaces.
- **LEASE:** A rental agreement between one property, one tenant, and one responsible agent, with a start date, end date, monthly rent, and status. A property can have many leases over time, but only one active lease at a time (BR-14 to BR-18).
- **PAYMENT:** A money record that belongs to one lease, such as rent or deposit. It is kept as history and cannot be hard-deleted (BR-19 to BR-22).
- **MAINTENANCE_REQUEST:** A repair request raised by the tenant or agent of an active lease, with a description, report date, cost, and status. While a property is under maintenance, it cannot be leased (BR-19, BR-20).

2.1.2 RELATIONSHIP DEFINITIONS

Relationship	Cardinality	Description
PERSON — is a — OWNER / TENANT / AGENT	1:1 (ISA)	Every OWNER, TENANT, and AGENT is a specialization of PERSON, inheriting name, phone, and email
PROPERTY — is a — RESIDENTIAL_PROPERTY / COMMERCIAL_PROPERTY	1:1 (ISA)	Every property is specialized as either a residential or a commercial property, each with its own extra attributes.
OWNER — owns — PROPERTY	1:N	One owner may own many properties, but each property has exactly one owner
AGENT — manages — PROPERTY	1:N	One agent may manage many properties, but each property has exactly one responsible agent at a time
PROPERTY — leased via — LEASE	1:N	One property may have many leases over time, but only one active lease at a time; each lease is for exactly one property
TENANT — signs — LEASE	1:N	One tenant may sign many leases, but each lease is signed by exactly one tenant
AGENT — handles — LEASE	1:N	One agent may handle many leases, but each lease is handled by exactly one responsible agent
LEASE — has — PAYMENT	1:N	One lease may generate many payment records, but each payment belongs to exactly one lease

Relationship	Cardinality	Description
LEASE — submits — MAINTENANCE_REQUEST	1:N	One lease may give rise to many maintenance requests, but each request references exactly one lease

2.2 Entity and Attribute Definitions (Data Dictionary)

This section defines every entity, its attributes, keys, and business-rule constraints, following ISO/IEC 11179 metadata conventions. Primary-key rows are shown in bold; foreign-key attributes are italicized.

2.2.1 Table: PERSON (Supertype)

Attribute	Data Type	Key	Null?	Default	Business Rules & Constraints
PersonID	INT	PK	No	None	Unique identifier of a person.
FullName	VARCHAR(100)	None	No	None	Legal full name of the person.
Phone	VARCHAR(20)	None	Yes	NULL	Contact phone number.
Email	VARCHAR(100)	None	Yes	NULL	Contact email address.

2.2.2 Table: OWNER (Subtype)

Attribute	Data Type	Key	Null?	Default	Business Rules & Constraints
PersonID	INT	PK, FK	No	None	References PERSON(PersonID); uniquely identifies the owner.
Address	VARCHAR(255)	None	No	None	Owner's mailing address.

2.2.3 Table: TENANT (Subtype)

Attribute	Data Type	Key	Null?	Default	Business Rules & Constraints
PersonID	INT	PK, FK	No	None	References PERSON(PersonID); uniquely identifies the tenant.
IDCardNumber	VARCHAR(20)	AK	No	None	Unique identification card number (BR-10).

2.2.4 Table: AGENT (Subtype)

Attribute	Data Type	Key	Null?	Default	Business Rules & Constraints
PersonID	INT	PK, FK	No	None	References PERSON(PersonID); uniquely identifies the agent.
CommissionRate	DECIMAL(5,2)	None	No	None	Commission rate applied to closed transactions.

2.2.5 Table: PROPERTY

Attribute	Data Type	Key	Null?	Default	Business Rules & Constraints
PropertyID	INT	PK	No	None	Unique identifier of the property (BR-01).
Title	VARCHAR(100)	None	No	None	Listing title of the property.
Address	VARCHAR(255)	None	No	None	Physical address of the property.
PropertyType	VARCHAR(20)	None	No	None	Classification, e.g. RESIDENTIAL / COMMERCIAL.
ListingType	VARCHAR(20)	None	No	None	CHECK IN ('FOR_SALE','FOR_RENT') — one purpose only (BR-02).
Price	DECIMAL(15,2)	None	No	None	Listing price: CHECK (Price > 0).
Status	VARCHAR(20)	None	No	'AVAILABLE'	CHECK IN ('AVAILABLE','LEASED','UNDER_MAINTENANCE') (BR-03).
OwnerID	INT	FK	No	None	<i>References OWNER(PersonID) — exactly one owner (BR-01).</i>
AgentID	INT	FK	No	None	<i>References AGENT(PersonID) — responsible agent (BR-06).</i>

2.2.6 Table: LEASE

Attribute	Data Type	Key	Null?	Default	Business Rules & Constraints
LeaseID	INT	PK	No	None	Unique identifier of the lease.
StartDate	DATE	None	No	None	Lease start date (BR-16).
EndDate	DATE	None	No	None	CHECK (EndDate > StartDate) (BR-16).
MonthlyRent	DECIMAL(15,2)	None	No	None	CHECK (MonthlyRent > 0) (BR-16).
DepositAmount	DECIMAL(15,2)	None	No	None	Security deposit amount.
Status	VARCHAR(20)	None	No	'PENDING'	Lifecycle status of the lease (BR-15, BR-17, BR-18).
<i>PropertyID</i>	<i>INT</i>	<i>FK</i>	<i>No</i>	<i>None</i>	<i>References PROPERTY(PropertyID) — at most one active lease (BR-15).</i>
<i>TenantID</i>	<i>INT</i>	<i>FK</i>	<i>No</i>	<i>None</i>	<i>References TENANT(PersonID) — the tenant (BR-14).</i>
<i>AgentID</i>	<i>INT</i>	<i>FK</i>	<i>No</i>	<i>None</i>	<i>References AGENT(PersonID) — the agent responsible for this lease (BR-10).</i>

2.2.7 Table: PAYMENT

Attribute	Data Type	Key	Null?	Default	Business Rules & Constraints
PaymentID	INT	PK	No	None	Unique identifier of the payment transaction.
PaymentDate	DATE	None	No	None	Date the payment was recorded (BR-20).

Attribute	Data Type	Key	Null?	Default	Business Rules & Constraints
Amount	DECIMAL(15,2)	None	No	None	CHECK (Amount > 0) (BR-20).
PaymentType	VARCHAR(20)	None	No	None	e.g. RENT, DEPOSIT, FEE.
PaymentMethod	VARCHAR(50)	None	Yes	NULL	Payment channel used.
<i>LeaseID</i>	<i>INT</i>	<i>FK</i>	<i>No</i>	<i>None</i>	<i>References LEASE(LeaseID) — every record ties to one lease (BR-19, BR-21, BR-22).</i>

2.2.8 Table: MAINTENANCE_REQUEST

Attribute	Data Type	Key	Null?	Default	Business Rules & Constraints
RequestID	INT	PK	No	None	Unique identifier of the maintenance request.
Description	TEXT	None	No	None	Description of the reported issue (BR-23).
ReportDate	DATE	None	No	None	Date the request was reported (BR-23).
EstimatedCost	DECIMAL(15,2)	None	Yes	NULL	Estimated repair cost.
ActualCost	DECIMAL(15,2)	None	Yes	NULL	Actual repair cost once completed.
Status	VARCHAR(20)	None	No	'PENDING'	Lifecycle status of the request (BR-24).
<i>LeaseID</i>	<i>INT</i>	<i>FK</i>	<i>No</i>	<i>None</i>	<i>References LEASE(LeaseID) — lease from which the request arises.</i>

2.2.9 Table: RESIDENTIAL_PROPERTY (Subtype of PROPERTY)

Attribute	Data Type	Key	Null?	Default	Business Rules & Constraints
PropertyID	INT	PK, FK	No	None	References PROPERTY(PropertyID) ; uniquely identifies the residential unit.
Bedrooms	INT	None	No	None	Number of bedrooms.

Attribute	Data Type	Key	Null?	Default	Business Rules & Constraints
Bathrooms	INT	None	No	None	Number of bathrooms.
Area	DECIMAL	None	No	None	Total usable living area.
FurnishedStatus	VARCHAR(20)	None	Yes	NULL	e.g. Fully Furnished, Unfurnished.

2.2.10 Table: COMMERCIAL_PROPERTY (Subtype of PROPERTY)

Attribute	Data Type	Key	Null?	Default	Business Rules & Constraints
PropertyID	INT	PK, FK	No	None	References PROPERTY(PropertyID); uniquely identifies the commercial unit.
BusinessType	VARCHAR(50)	None	Yes	NULL	e.g. Office, Retail, Warehouse.
FloorArea	DECIMAL(10,2)	None	No	None	Total floor area available for commercial use.
ParkingSpaces	INT	None	No	0	Number of available vehicle parking spaces.

2.3 Logical Schema Mapping

Based on the conceptual model in Section 2.1, the schema consists of 10 relations. Each entity becomes one relation; each 1:N relationship is implemented by placing the primary key of the 1-side as a foreign key on the N-side; each ISA relationship is implemented by giving the subtype a primary key that is also a foreign key referencing its supertype. No relationship requires a separate relation, so the schema contains no junction tables and no redundant attributes.

2.3.1 Relational Schema

```
PERSON(
  PersonID PK,
  FullName,
  Phone,
  Email
)
```

```
OWNER(
  PersonID PK, FK → PERSON(PersonID),
  Address
)
```

```
TENANT(
  PersonID PK, FK → PERSON(PersonID),
  IDCardNumber UK
)
```

```

AGENT(
  PersonID PK, FK → PERSON(PersonID),
  CommissionRate
)

PROPERTY(
  PropertyID PK,
  Title,
  Address,
  PropertyType,
  ListingType,
  Price,
  Status,
  OwnerID FK → OWNER(PersonID),
  AgentID FK → AGENT(PersonID)
)

RESIDENTIAL_PROPERTY(
  PropertyID PK, FK → PROPERTY(PropertyID),
  Bedrooms,
  Bathrooms,
  Area,
  FurnishedStatus
)

COMMERCIAL_PROPERTY(
  PropertyID PK, FK → PROPERTY(PropertyID),
  BusinessType,
  FloorArea,
  ParkingSpaces
)

LEASE(
  LeaseID PK,
  StartDate,
  EndDate,
  MonthlyRent,
  DepositAmount,
  Status,
  PropertyID FK → PROPERTY(PropertyID),
  TenantID FK → TENANT(PersonID),
  AgentID FK → AGENT(PersonID)
)

PAYMENT(
  PaymentID PK,
  PaymentDate,
  Amount,
  PaymentType,
  PaymentMethod,
  LeaseID FK → LEASE(LeaseID)
)

MAINTENANCE_REQUEST(
  RequestID PK,
  Description,
  ReportDate,
  EstimatedCost,
  ActualCost,
  Status,
  LeaseID FK → LEASE(LeaseID)
)

```

2.3.2 Mapping of Entities to Relations

Conceptual Entity	Relational Table	Primary Key
PERSON	PERSON	PersonID
OWNER	OWNER	PersonID
TENANT	TENANT	PersonID
AGENT	AGENT	PersonID
PROPERTY	PROPERTY	PropertyID
RESIDENTIAL_PROPERTY	RESIDENTIAL_PROPERTY	PropertyID
COMMERCIAL_PROPERTY	COMMERCIAL_PROPERTY	PropertyID
LEASE	LEASE	LeaseID
PAYMENT	PAYMENT	PaymentID
MAINTENANCE_REQUEST	MAINTENANCE_REQUEST	RequestID

PERSON is mapped to the PERSON relation, while OWNER, TENANT, and AGENT are mapped as subtype relations whose primary key is also a foreign key referencing PERSON(PersonID), preserving the ISA structure of the EER model. The same pattern applies to PROPERTY and its two subtypes.

2.3.3 Mapping of Relationships

Relationship	Foreign Key Implementation
PERSON — is a — OWNER / TENANT / AGENT	OWNER.PersonID, TENANT.PersonID, AGENT.PersonID → PERSON.PersonID (PK is also FK)
PROPERTY — is a — RESIDENTIAL_PROPERTY / COMMERCIAL_PROPERTY	RESIDENTIAL_PROPERTY.PropertyID, COMMERCIAL_PROPERTY.PropertyID → PROPERTY.PropertyID (PK is also FK)
OWNER — owns — PROPERTY	PROPERTY.OwnerID → OWNER.PersonID
AGENT — manages — PROPERTY	PROPERTY.AgentID → AGENT.PersonID
PROPERTY — leased via — LEASE	LEASE.PropertyID → PROPERTY.PropertyID
TENANT — signs — LEASE	LEASE.TenantID → TENANT.PersonID
AGENT — handles — LEASE	LEASE.AgentID → AGENT.PersonID
LEASE — has — PAYMENT	PAYMENT.LeaseID → LEASE.LeaseID
LEASE — submits — MAINTENANCE_REQUEST	MAINTENANCE_REQUEST.LeaseID → LEASE.LeaseID

2.3.4 Referential Integrity

Foreign key constraints maintain referential integrity between related tables. For example, PROPERTY.OwnerID references OWNER.PersonID, so a property cannot reference an owner that does not exist. The same principle applies to all other foreign keys in the schema, so the logical schema preserves the structural relationships defined by the conceptual model while ensuring consistency between related relations. Foreign keys are declared with ON DELETE RESTRICT so that leases, payments, and maintenance records can never be orphaned or hard-deleted (BR-17, NFR-04).

2.4 Normalization Verification

The logical schema is analyzed against First (1NF), Second (2NF), Third (3NF), and Boyce-Codd (BCNF) normal forms to reduce redundancy and avoid update, insertion, and deletion anomalies. The verification is based on the functional dependencies implied by the current database design.

2.4.1 First Normal Form (1NF)

A relation satisfies 1NF when every attribute contains a single atomic value, there are no repeating groups, and each record can be uniquely identified by a key. The proposed relations satisfy these conditions: PERSON(PersonID, FullName, Phone, Email) has a single value per attribute for each person, and the type-specific attributes of a property are not stored as repeating or optional columns inside PROPERTY — they are placed in the separate subtype relations RESIDENTIAL_PROPERTY and COMMERCIAL_PROPERTY.

2.4.2 Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and every non-key attribute is fully functionally dependent on the entire primary key. Every relation in the proposed schema has a single-attribute primary key (PersonID, PropertyID, LeaseID, PaymentID, RequestID), so a partial dependency on a subset of the primary key cannot occur. All relations therefore satisfy 2NF.

2.4.3 Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and contains no transitive dependency of a non-key attribute on the primary key. For every relation in the schema, the non-key attributes depend directly on the primary key with no attribute determining another non-key attribute:

PERSON: PersonID $\rightarrow$ FullName, Phone, Email

OWNER: PersonID $\rightarrow$ Address

TENANT: PersonID $\rightarrow$ IDCardNumber

AGENT: PersonID $\rightarrow$ CommissionRate

PROPERTY: PropertyID $\rightarrow$ Title, Address, PropertyType, ListingType, Price, Status, OwnerID, AgentID

LEASE: LeaseID $\rightarrow$ StartDate, EndDate, MonthlyRent, DepositAmount, Status, PropertyID, TenantID, AgentID

PAYMENT: PaymentID $\rightarrow$ PaymentDate, Amount, PaymentType, PaymentMethod, LeaseID

MAINTENANCE_REQUEST: RequestID $\rightarrow$ Description, ReportDate, EstimatedCost, ActualCost, Status, LeaseID

RESIDENTIAL_PROPERTY: PropertyID $\rightarrow$ Bedrooms, Bathrooms, Area, FurnishedStatus

COMMERCIAL_PROPERTY: PropertyID $\rightarrow$ BusinessType, FloorArea, ParkingSpaces

Since no such transitive dependency exists in any relation, all ten relations satisfy 3NF.

2.4.4 BCNF Verification

A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$, the determinant X is a superkey. For the proposed schema, the principal dependencies are determined by the primary keys — PropertyID, LeaseID, and so on are each a superkey of their own relation, and no other determinant appears. Therefore, under the functional dependencies defined by the current system specification, all ten relations — including RESIDENTIAL_PROPERTY and COMMERCIAL_PROPERTY, where PropertyID is a candidate key — satisfy BCNF.